# 2026 CSP-S 初赛真题详解与答案速查

**一、单项选择题**

| 题号 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 答案 | D | D | C | D | A |
| 题号 | 6 | 7 | 8 | 9 | 10 |
| 答案 | C | A | B | A | D |
| 题号 | 11 | 12 | 13 | 14 | 15 |
| 答案 | C | C | B | C | B |

**二、阅读程序**

| 题号 | 16 | 17 | 18 | 19 | 20 | 21 |
|---|---|---|---|---|---|---|
| 答案 | ✓ | ✓ | × | C | B | C |
| 题号 | 22 | 23 | 24 | 25 | 26 | 27 |
| 答案 | ✓ | ✓ | × | B | B | D |
| 题号 | 28 | 29 | 30 | 31 | 32 | 33 |
| 答案 | ✓ | × | × | A | C | C |

**三、完善程序**

| 题号 | 34 | 35 | 36 | 37 | 38 |
|---|---|---|---|---|---|
| 答案 | C | D | B | A | C |
| 题号 | 39 | 40 | 41 | 42 | 43 |
| 答案 | C | B | D | A | A |

## 一、单项选择题（每题 2 分，共 30 分）

**答案**：1.D 2.D 3.C 4.D 5.A 6.C 7.A 8.B 9.A 10.D 11.C 12.C 13.B 14.C 15.B

**第 1 题** 执行下列代码后，`cnt` 的值是（ ）。

```
int x = 2026, cnt = 0;
while (x) {
    x &= x - 1;
```

```
    cnt++;
}
```

- A．6
- B．7
- C．11
- D．8

【解析】 `x &= x - 1` 的作用是清除 x 的最低一位 1（`x-1` 会把最低位的 1 变 0、其后的 0 全变 1，再与 x 相与即可抹掉最低位 1）。因此循环次数就是 `x` 的二进制中 1 的个数。
$2026 = 11111101010_2$，其中 1 的个数为 8（$1024 + 512 + 256 + 128 + 64 + 32 + 8 + 2 = 2026$）。正确选项：D

**第 2 题** 用权值 {1, 2, 3, 4, 5, 6, 7, 8} 构造哈夫曼树，其带权路径长度是（ ）。

- A．108
- B．96
- C．99
- D．102

【解析】哈夫曼算法每次取两个最小权值合并，合并代价累加即为带权路径长度：$1+2=3$；$3+3=6$；$4+5=9$；$6+6=12$；$7+8=15$；$9+12=21$；$15+21=36$。合计 $3+6+9+12+15+21+36=102$。正确选项：D

**第 3 题** 把 1 到 1000 的所有整数按十进制写出，数字“1”总共出现了多少次（ ）。

- A．300
- B．271
- C．301
- D．320

信奥陈老师

【解析】先看 $1\sim999$，把它们都补齐成三位（含前导零，如 007），共 1000 个数。每一位上取 0～9 的机会均等，故每一位是 1 的有 $1000\div10=100$ 个，三位合计 300 次。再加上 1000 中的那一个 1，共 301 次。（补前导零不影响计数，因为 0 不是 1。）正确选项：C

**第 4 题** 将 5 封信随机装入 5 个写好地址的信封（每封一个），恰好有 2 封装对的方案数是（ ）。

- A．44
- B．24
- C．10
- D．20

【解析】先选出装对的 2 封：$\binom{5}{2}=10$ 种；剩下 3 封必须**全部装错**，即 3 个元素的错排数 $D_3=3!\left(\frac{1}{0!}-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}\right)=6-6+3-1=2$。由乘法原理，方案数为 $10\times2=20$。（$D_3$ 也可直接枚举：两个错排即两个 3-轮换。）正确选项：D

**第 5 题** 的值是（ ）。

- A．29
- B．9
- C．43
- D．81

【解析】依次求 3 的幂对 100 取模：$3^1=3,\ 3^2=9,\ 3^3=27,\ 3^4=81,\ 3^5=43,\ 3^6=29,\ldots$，继续算到 $3^{20}\equiv1\pmod{100}$，故循环节为 20。$2026=20\times101+6$，所以 $3^{2026}\equiv3^6\equiv29\pmod{100}$。正确选项：A

**第 6 题** 有 5 堆石子排成一行，重量依次为 4、1、3、2、5。每次只能把相邻的两堆合并成一堆，代价为这两堆重量之和。将所有石子合并成一堆的最小总代价是（ ）。

- A．36
- B．35
- C．34

- D．33

【解析】这是**相邻**合并问题，不能直接套哈夫曼（哈夫曼允许任意两堆合并），要用区间 DP。设 $dp[i][j]$ 为把第 $i$ 到第 $j$ 堆合并成一堆的最小代价，转移为

$$dp[i][j]=\sum_{k=i}^{j}a_k+\min_{i\le m<j}\{dp[i][m]+dp[m+1][j]\}$$ 前缀和依次为 $0,4,5,8,10,15$。计算：长度 2：$dp[1][2]=5$，$dp[2][3]=4$，$dp[3][4]=5$，$dp[4][5]=7$；长度 3：$dp[1][3]=8+4=12$，$dp[2][4]=6+4=10$，$dp[3][5]=10+5=15$；长度 4：$dp[1][4]=10+10=20$，$dp[2][5]=11+10=21$；长度 5：$dp[1][5]=15+\min\{21,\ 5+15,\ 12+7,\ 20\}=15+19=34$。正确选项：C

**第 7 题** 树状数组维护长度 的序列，查询前缀和 `sum(11)` 与单点修改 `add(3, x)` 分别需要访问树状数组中多少个下标（ ）。

- A．3 和 4
- B．4 和 4
- C．3 和 5
- D．4 和 3

【解析】树状数组（下标从 1 开始）：`sum(i)` 每次 `i -= i & -i`：$11\to10\to8\to0$，访问 11、10、8，共 3 个。`add(i)` 每次 `i += i & -i` 直到超过 $n$：$3\to4\to8\to16\to32$（越界停止），访问 3、4、8、16，共 4 个。正确选项：A

**第 8 题** 有向无环图 顶点集为，边集为，顶点 4 与任何顶点均不相邻。该图不同的拓扑序共有多少种（ ）。

- A．12
- B．8
- C．4
- D．6

陈老师

【解析】约束只有「1 必须排在 2 和 3 之前」，顶点 4 完全自由。先排 $\{1,2,3\}$：满足 1 在最前的排列有 $1,2,3$ 与 $1,3,2$ 两种；再把 4 插入到 4 个位置中的任意一处，共 4 种。总数 $2\times 4=8$。正确选项：B

- 

**第 9 题** 某分治算法满足 $T(n)=T(n/3)+T(2n/3)+\Theta(n)$，$T(1)=O(1)$，则 $T(n)$ 是（）。

- A．$\Theta(n\log n)$
- B．$\Theta(n^2)$
- C．$\Theta(n^{1.5})$
- D．$\Theta(n)$

【解析】递归树中，每一层的子问题规模之和恒为 $n$，因此每层的工作量都是 $\Theta(n)$。递归深度由 $n\left(\frac{2}{3}\right)^d=1$ 决定，即 $d=\log_{3/2}n=O(\log n)$。故 $T(n)=\Theta(n\log n)$。正确选项：A

**第 10 题** 无根树含 9 个结点（编号为 1—9），边集为 $\{(1,2),(1,3),(2,4),(2,5),(3,6),(6,7),(7,8),(5,9)\}$。该树的直径（以边数计）与重心分别是（）。

- A．直径 6，重心为结点 3
- B．直径 7，重心为结点 2
- C．直径 8，重心为结点 1
- D．直径 7，重心为结点 1

【解析】树的形状：1 连 2、3；2 连 4、5；5 连 9；3 连 6；6 连 7；7 连 8。最长路径为 $8-7-6-3-1-2-5-9$，共 7 条边（也可取 $4-2-1-3-6-7-8$，同为 7），故直径为 7。重心：以 1 为根，子树 $\{2,4,5,9\}$ 大小 4、子树 $\{3,6,7,8\}$ 大小 4，都不超过 $\lfloor 9/2\rfloor=4$，故结点 1 是重心。再看结点 2：去掉 2 后含 $\{1,3,6,7,8\}$ 的分量大小为 5，超过 4，不是重心。正确选项：D

**第 11 题** 一张有向图缩点后得到的有向无环图含 6 个顶点，其中入度为 0 的顶点有 3 个，出度为 0 的顶点有 4 个。为使原图变成强连通图，至少需要添加多少条有向边（）。

- A．7
- B．6
- C．4
- D．3

【解析】经典结论：把 DAG 通过加边变成强连通，最少需要 $\max(p, q)$ 条边，其中 $p$ 为入度为 0 的顶点数（源点数）、$q$ 为出度为 0 的顶点数（汇点数）。这里 $\max(3, 4) = 4$。构造方法：给 3 个源点各引一条**来自汇点**的入边（同时用掉 3 个汇点的出边），再给剩下的那 1 个汇点补一条**来自源点**的入边，共 4 条。正确选项：C

**第 12 题** 含 6 个结点的不同形态的二叉树共有多少棵（结点不带标号，区分左右子树）（ ）。

- A．42
- B．429
- C．132
- D．720

【解析】$n$ 个结点的不同形态二叉树数目就是卡特兰数 $C_n = \frac{1}{n+1}\binom{2n}{n}$。$C_6 = \frac{1}{7}\binom{12}{6} = \frac{924}{7} = 132$。正确选项：C

**第 13 题** 字符串，其所有既是真前缀又是真后缀的子串（非空）的长度之和是（ ）。

- A．4
- B．6
- C．7
- D．5

【解析】$S$ 长度为 9，逐一看长度 1～8 的前缀与同长度后缀是否相同：长度 1：`a` vs `b` ✗；长度 2：`ab` vs `ab` ✓；长度 3：`aba` vs `bab` ✗；长度 4：`abab` vs `abab` ✓；长度 5～8 均不相同。所以边界串长度为 2 和 4，长度之和 $2 + 4 = 6$。正确选项：B

**第 14 题** 用归并排序统计逆序对。若把判断条件中的 `a[i] <= a[j]` 改成 `a[i] < a[j]`，则 `ans` 统计出的结果（ ）。

- A．完全不变
- B．变为原来的两倍

信奥陈老师

- C．变为满足 $i<j$ 且="" $a[i]\ge="" a[j]$="" 的数对个数<="" section="">
- D．变为原来的一半

【解析】原写法在 $a[i] \le a[j]$ 时取左段，否则（即 $a[i] > a[j]$）取右段并累加 `mid-i+1`，统计的是严格逆序对 $a[i] > a[j]$。改成 `a[i] < a[j]` 后，$a[i] = a[j]$ 时改走 else 分支并累加，于是把相等的情形也算了进去，统计的变为 $a[i] \ge a[j]$ 的数对个数（即非严格逆序对）。这也说明：想统计严格逆序对必须用 `<=`，想统计非严格逆序对用 `<`。正确选项：C

**第 15 题** 执行 `power(2, 100, 1000)` 调用下列函数，返回值是（ ）。

```
long long power(long long a, long long b, long long p) {
    long long r = 1 % p;
    while (b) {
        if (b & 1)
            r = r * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return r;
}
```

- A．576
- B．376
- C．976
- D．176

---

【解析】即求 $2^{100} \bmod 1000$，用快速幂逐次平方：$2^{10} \equiv 24$，$2^{20} \equiv 24^2 = 576$，$2^{40} \equiv 576^2 = 331776 \equiv 776$，$2^{80} \equiv 776^2 = 602176 \equiv 176$。于是 $2^{100} = 2^{80} \cdot 2^{20} \equiv 176 \times 576 = 101376 \equiv 376 \pmod{1000}$。正确选项：B

---

## 二、阅读程序（判断题每题 1.5 分，选择题每题 3 分，共 40 分）

信奥陈老师

（1）

```
#include <iostream>
#include <string>
using namespace std;
int a[100];
string s;
int gen[13] = {1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1};
int main() {
    cin >> s;
    for (int i = 0; i < 32; ++i) {
        a[i] = s[i] - '0';
    }
    for (int i = 32; i < 44; ++i) {
        a[i] = 0;
    }
    for (int i = 0; i < 32; ++i) {
        if (a[i] == 0) continue;
        for (int j = 0; j < 13; ++j) {
            a[i + j] ^= gen[j];
        }
    }
    for (int i = 32; i < 44; ++i) {
        cout << a[i];
    }
    cout << endl;
    return 0;
}
```

信奥陈老师

**程序先读明白**：`a[0..31]` 存放输入的 32 位串，`a[32..43]` 补 12 个 0，长度变成 44 位。外层 `i` 从左到右扫描，一旦 `a[i]` 为 1，就把 13 位的 `gen` 对齐到 `i` 做一次异或（模 2 减法），把 `a[i]` 消成 0。这正是**模 2 多项式除法（CRC）**的过程：`gen = 1100000001111` 是生成多项式，最终 `a[32..43]` 就是 12 位的余数。

**判断题**

**第 16 题**（1 分）当输入为 32 个 `'0'` 时，程序输出 12 个 0。

【解析】输入全 0 时 a[0..31] 全为 0，外层循环每次都因 a[i] == 0 而 continue，异或步骤一次都不执行；a[32..43] 仍为 0。输出 12 个 0。正确判断：√

**第 17 题** 程序运行结束后，数组 a 中下标从 0 到 31 的元素一定全部为 0。

【解析】处理到位置 $i$ 时，若 a[i] 为 1，则 a[i] ^= gen[0] 即 a[i] ^= 1，把它清零（gen[0] = 1）。之后的步骤只会影响下标 $\geq i+1$ 的位置，不会再改动 i。因此循环结束时 a[0..31] 全为 0。正确判断：√

**第 18 题** 若将第 12—14 行（为 a[32] 到 a[43] 补 0 的循环）删除，会改变程序输出的结果。

【解析】a 是全局数组，未显式初始化的元素默认为 0，所以 a[32..43] 本来就已经是 0。这个补 0 循环只是显式重复了一次，删掉后结果完全不变。正确判断：×

**单选题**

**第 19 题** 关于第 6 行定义的数组 gen，下列说法正确的是（ ）。

- A．gen 共有 12 个元素，表示一个 12 位的除数
- B．gen 共有 13 个元素，表示一个 13 位的被除数
- C．gen 共有 13 个元素，其中 gen[0] 是除数的最高位
- D．gen 共有 13 个元素，其中 gen[12] 是除数的最高位

【解析】初值表里有 13 个数，故 gen 有 13 个元素；在 a[i + j] ^= gen[j] 中，gen[0] 与当前最高位 a[i] 对齐，所以 gen[0] 对应除数的最高位。A 元素个数错；B 说成"被除数"错；D 把最高位说反了。正确选项：C

**第 20 题** 该程序实现的功能，最准确的说法是（ ）。

- A．将输入的 32 位串看成二进制数，输出与 13 位二进制数 1100000001111 按位异或的结果
- B．将输入串视为 32 位二进制数，在其后补 12 个 0（即计算），再对它做模 2 除法求余数，并输出 12 位余数
- C．对输入的 32 位串逐位取反并输出结果

信奥陈老师

- D．统计输入串中 1 的个数，并把该个数用 12 位二进制表示后输出

【解析】补 12 个 0 相当于把 $M$ 左移 12 位（乘 $2^{12}$），随后的循环就是拿生成多项式 `gen` 对它做模 2 除法，剩下的 `a[32..43]` 正是 12 位余数——这就是 CRC-12 校验码的计算。A、C、D 都与程序的循环结构不符。正确选项：B

**第 21 题** 若把第 16 行 `if (a[i] == 0) continue;` 删除，说法正确的是（ ）。

- A．程序输出的结果不会改变
- B．可能造成程序运行错误
- C．程序能够正常输出一个 12 位 `'0'` / `'1'` 串，但是输出结果与输入的 s 无关
- D．程序运行结束后，a[0] 的值一定为 0

【解析】删掉 `continue` 后，无论 `a[i]` 是 0 还是 1，每轮都会固定地异或一次 `gen`。此时执行的操作序列只由 `i` 决定，与输入完全无关，因此输出是一个固定串，不再依赖 `s`。由于全程只有按位异或，`a[32..43]` 始终是 0/1，所以仍能正常输出 12 位 `'0'` / `'1'` 串。C 正确。A 错（结果会变）；B 错（下标最大到 $31+12=43$，不会越界）；D 错（第 0 位会被异或成 1）。正确选项：C

（2）

```
#include <iostream>
using namespace std;
int n, m, a[100007], L, R, lg[100007], i, j, t, dp[100007][25], pw[25];
int gcd(int x, int y) {
    if (y == 0) return x;
    return gcd(y, x % y);
}
int main() {
    cin >> n >> m;
    for (i = 1; i <= n; i++) cin >> a[i];
    t = 0;
    pw[0] = 1;
    for (i = 1; i <= 24; i++) pw[i] = pw[i - 1] * 2;
    for (i = 1; i <= 100000; i++)
        if (pw[t + 1] >= i) lg[i] = t;
        else { t++; lg[i] = t; }
```

```
    for (i = 1; i <= n; i++) {
        dp[i][0] = a[i];
    }
    for (j = 1; j <= lg[n]; j++) {
        for (i = 1; i + pw[j] - 1 <= n; i++) {
            dp[i][j] = gcd(dp[i][j - 1], dp[i + pw[j - 1]][j - 1]);
        }
    }
    for (i = 1; i <= m; i++) {
        cin >> L >> R;
        cout << gcd(dp[L][lg[R - L + 1]], dp[R - pw[lg[R - L + 1]] + 1][lg[R -
L + 1]]) << endl;
    }
    return 0;
}
```

**程序先读明白**：这是求区间最大公约数的 **ST 表（稀疏表）**。`dp[i][j]` 表示从 `a[i]` 开始连续 $2^j$ 个数的 gcd；`lg[x]` 是满足 $2^t < x \le 2^{t+1}$ 的那个 $t$，即 $\lceil \log_2 x \rceil - 1$。查询时用两个长度为 $2^{lg}$ 的块覆盖区间 $[L,R]$，两块的 gcd 即为答案（gcd 可重复计算，重叠不影响结果）。

**判断题**

**第 22 题** 当，`a={4,2,6,3,3}`，且仅有一次查询 时，输出为 1。

【解析】区间 $a[2..5]=\{2,6,3,3\}$：$\gcd(2,6)=2$，$\gcd(2,3)=1$，$\gcd(1,3)=1$。所以输出 1。正确判断：√

**第 23 题** 当某次查询的区间长度为 1（即 ）时，该次查询的输出一定等于 `a[L]`。

【解析】长度为 1 时 `lg[1] = 0`（因为 $2^1 \ge 1$），查询式子变成 $\gcd(dp[L][0], dp[R-1+1][0]) = \gcd(a[L], a[L]) = a[L]$。正确判断：√

**第 24 题** 任意一次查询的输出结果一定不小于该查询区间内的最小值。

【解析】区间 gcd 一定是区间内每个数的约数，因此**不大于**区间最小值（说反了）。例如区间 $\{2,3\}$ 的最小值是 2，而 $\gcd=1<2$。正确判断：×

**单选题**

**第 25 题** 对于，数组 `dp[i][j]` 保存的是（ ）。

- A．从 a[i] 开始连续 个数的最大公约数
- B．从 a[i] 开始连续 个数的最大公约数

- C．a[i] 与 a[j] 的最大公约数
- D．从 a[1] 到 a[i] 的最大公约数
-

【解析】由转移 `dp[i][j] = gcd(dp[i][j-1], dp[i + 2^(j-1)][j-1])`，两个半段各长 $2^{j-1}$，拼起来正好 $2^j$ 个数。 正确选项：B

**第 26 题** 若把一次求最大公约数的运算视为 $O(1)$，则第 17—22 行建表过程的时间复杂度为（）。

- A．$\Theta(n)$
- B．$\Theta(n\log n)$
- C．$\Theta(n^2)$
- D．$\Theta(mn)$

【解析】外层 $j$ 有 $O(\log n)$ 层，内层 $i$ 的总量是 $O(n)$，故总复杂度 $O(n\log n)$。D 含查询次数 $m$，与建表无关。 正确选项：B

**第 27 题** 设 $x$ 为一次查询的区间长度（即 $x = R - L + 1$），则使得 `lg[x] = 5` 的 $x$ 的取值范围是（）。

- A．$[16, 31]$
- B．$[17, 32]$
- C．$[32, 63]$
- D．$[33, 64]$

【解析】由 `if (pw[t+1] >= i) lg[i] = t; else { t++; lg[i] = t; }` 可知 `lg[x] = t` 当且仅当 $2^t < x \le 2^{t+1}$。取 $t = 5$ 得 $32 < x \le 64$，即 $x \in [33, 64]$。 正确选项：D

（3）

```
#include <iostream>
using namespace std;
int n, fa[100007], f[100007], ans;
int main() {
    cin >> n;
    for (int i = 2; i <= n; ++i) {
        cin >> fa[i];
    }
    for (int i = n; i >= 2; --i) {
```

```
        if (f[fa[i]] + f[i] + 1 > ans) {
            ans = f[fa[i]] + f[i] + 1;
        }
        if (f[i] + 1 > f[fa[i]]) {
            f[fa[i]] = f[i] + 1;
        }
    }
    cout << ans << endl;
    return 0;
}
```

**程序先读明白**：因为 $fa[i] < i$，倒序枚举 $i$ 时所有儿子都已处理完。`f[v]` 表示以 $v$ 为根的子树中，从 $v$ 向下走的最长路径长度（高度）。每个结点合并「当前最长子链」与「新来的子链」来更新直径候选，因此 `ans` 最终是**树的直径**（以边数计）。

**判断题**

**第 28 题** 当，`fa[2]~fa[5]={1,2,3,4}` 时，程序输出 4。

【解析】这是一条链 $1-2-3-4-5$，直径即 4 条边。逐步跟踪：处理 5 时 `ans=1, f[4]=1`；处理 4 时 `ans=2, f[3]=2`；处理 3 时 `ans=3, f[2]=3`；处理 2 时 `ans=4, f[1]=4`。输出 4。正确判断：√

**第 29 题** 程序输出前，`f[1]` 的值一定等于 `ans` 的值。

【解析】不一定。反例：$n=4$，`fa[2]=1, fa[3]=1, fa[4]=2`（呈"Y"形）。此时直径为 $3-1-2-4$ 共 3 条边，而 `f[1]` 是 1 到最深叶子的距离，等于 2。逐步跟踪可得 `ans=3`、`f[1]=2`，两者不等。只有当最长路径的一个端点恰好是根时二者才相等。正确判断：×

**第 30 题** 将第 10—12 行与第 13—15 行两个 `if` 语句的顺序交换后，程序输出结果不受影响。

【解析】交换后先更新 `f[fa[i]]`，再用更新过的 `f[fa[i]]` 去计算候选值 `f[fa[i]] + f[i] + 1`。此时 `f[fa[i]]` 已经被改成 `f[i] + 1`，候选值变成 $2f[i]+2$，等于**把同一条子链用了两次**，会算出偏大的错误直径。故输出会改变。正确判断：×

**单选题**

**第 31 题** 程序输出的 `ans` 表示的是（ ）。

- A．树中距离最远的两个结点之间路径所经过的边数

- B．根结点 1 到最远叶子结点之间路径所经过的边数
- C．树中叶子结点的个数
- D．所有结点的父结点编号之和

【解析】 ans 每次取「已扫描子树中的最长下行链 + 当前子树的最长下行链 + 1」，正是经过当前结点的最长路径，全局取最大即为树的直径（边数）。B 只是其中的特殊情形（高度）；C、D 显然不符。 正确选项：A

**第 32 题** 当， fa[2]~fa[7]={1,1,2,2,3,3} 时，输出为（ ）。

- A．2
- B．3
- C．4
- D．5

【解析】结构为：1 的孩子 2、3；2 的孩子 4、5；3 的孩子 6、7。 最长路径如 $4-2-1-3-6$，经过 4 条边；也可取 $4-2-1-3-7$、$5-2-1-3-6$ 等，均为 4。 逐步跟踪：$ans$ 被刷新为 1（$i=7$）、2（$i=6$）、4（$i=2$），中间几步不产生更大值，最终输出 4。 正确选项：C

**第 33 题** 当，满足输出为 9 的合法输入种类数为（ ）。

- A．0
- B．9
- C．256
- D．512

【解析】10 个结点直径为 9，当且仅当这棵树是一条链（路径）。 把路径按"离根 1 的距离"来看：从 1 出发沿路径向两侧走，标号必须递增（因为要求 $fa[i]

（左侧标号递减） $\cdots\ a_2\ a_1\ 1\ b_1\ b_2\ \cdots$ （右侧标号递增）

把 $\{2,\ldots,10\}$ 分成"1 左侧"与"1 右侧"两个集合即可确定路径，共 $2^9=512$ 种分法。 但把左右两侧整体互换会得到**同一条无根路径**（对应同一个 fa 数组），故每种树被数了两次，实际种类数为 $512\div 2=256$。 正确选项：C

## 三、完善程序（单选，每小题 3 分，共 30 分）

### （1）平衡路线

给定一张有 $n$ 个顶点，$m$ 条边的无向图，每条边带有符号 + 或 - 。对于一条从顶点 $s$ 到顶点 $t$ 的路线，允许重复经过顶点和边，定义一条路线的权值如下：记 $n^+$、$n^-$ 分别为经过的 + 边数和经过的 - 边数，则该路线的权值为 $|n^+ - n^-|$。

请计算从 $s$ 到 $t$ 的路线的最小权值，若不存在从 $s$ 到 $t$ 的路线，则输出 $-1$。

输入第一行为四个整数 $n, m, s, t$，接下来 $m$ 行，每行给出两个整数 $a, b$ 和一个字符 + 或 - ，描述一条连接 $a$ 与 $b$ 的无向边符号。

数据满足 $2 \le n \le 2 \times 10^5$，$1 \le m \le 4 \times 10^5$，$1 \le s, t \le n$ 且 $s \ne t$，$1 \le a, b \le n$，可能出现重边。

以下程序通过 BFS 求出最小权值，请补全程序。

信奥陈老师

```
#include <iostream>
constexpr int N = 200005;
constexpr int M = 400005;

int n, m, s, t;
int h[N], e[M << 1], ne[M << 1], w[M << 1], ids;
int d[N], q[N], c[N];

void add(int a, int b, int z) {
    e[ids] = b;
    w[ids] = z;
    ne[ids] = h[a];
    h[a] = ids++;
}

int main() {
    std::cin >> n >> m >> s >> t;
    for (int i = 1; i <= n; i++)
        h[i] = d[i] = c[i] = -1;
```

```
    for (int i = 0; i < m; i++) {
        int a, b;
        char op[2];
        std::cin >> a >> b >> op;
        int t =  ① ;
        add(a, b, t);
        add(b, a, t);
    }
    int hh = 0, tt = 0;
    int p = 0, ng = 0, ok = 1;
    q[tt++] = s;
    d[s] = c[s] = 0;

    while ( ② ) {
        int x = q[hh++];
        for (int i = h[x]; i != -1; i = ne[i]) {
            int y = e[i];
            if (w[i] > 0) p = 1;
            if (w[i] < 0) ng = 1;
            if (d[y] == -1) {
                d[y] =  ③ ;
                c[y] = c[x] ^ 1;
                q[tt++] = y;
            } else if ( ④ )
                ok = 0;
        }
    }
    if (d[t] == -1) {
        std::cout << -1;
        return 0;
    }
    if (!p || !ng) {
        std::cout << d[t];
        return 0;
    }
    if ( ⑤ ) std::cout << 0;
    else std::cout << 1;
    return 0;
}
```

信奥陈老师

给定一张有 $n$ 个顶点、$m$ 条边的无向图，每条边带有符号 `'+'` 或 `'-'`。求从 $s$ 到 $t$ 的路线的最小权值 $|n_+ - n_-|$（允许重复经过顶点和边）；若不可达输出 $-1$。

**程序先读明白**：给 `'+'` 边赋权 $+1$、`'-'` 边赋权 $-1$，则路线权值就是边权之和。BFS 时用 `c[]` 记录**距离奇偶性**（`c[y] = c[x] ^ 1`），用 `d[]` 记录最短边数；`p`、`ng` 标记是否出现过正权边与负权边。关键结论：一旦能访问到两种符号的边，就可以通过来回走某条边来成对地调整权值和（每来回一次改变 $\pm 2$），从而把权值调到 0 或 1。

**第 34 题** ①处应填（）。

- A. `op[0] == '+' ? 0 : 1`
- B. `op[0] == '+'`
- C. `op[0] == '+' ? 1 : -1`
- D. `op[0] == '-' ? 1 : 0`

【解析】后面用 `if (w[i] > 0) p = 1; if (w[i] < 0) ng = 1;` 判断边的符号，说明权值必须取 $\pm 1$，故填 `op[0] == '+' ? 1 : -1`。A、D 取 0/1，无法区分负权；B 是布尔值。正确选项：C

**第 35 题** ②处应填（ ）。

- A. `hh < n`
- B. `tt < n`
- C. `hh <= tt`
- D. `hh < tt`

【解析】手写队列用 `hh` 指向队首、`tt` 指向队尾后一位，队列非空的条件是 `hh < tt`。C 会在队列空时多取一次；A、B 与队列容量无关。正确选项：D

**第 36 题** ③处应填（ ）。

- A. `d[y] + 1`
- B. `d[x] + 1`
- C. `d[x]`
- D. `d[x] - 1`

信奥陈老师

【解析】BFS 求最短边数，从 x 走到 y 距离加一，故 d[y] = d[x] + 1 。正确选项：B

**第 37 题** ④处应填（ ）。

- A. c[y] == c[x]
- B. w[i] == 1
- C. c[y] != c[x]
- D. d[y] + 1 != d[x]

【解析】此处是"y 已被访问过"的分支。若 y 与 x 的奇偶标记相同，说明这条非树边构成奇环，即图不是二分图，于是存在两种奇偶性的通路，可以把权值调成 0。因此把 ok 置 0 的条件是 c[y] == c[x] 。正确选项：A

**第 38 题** ⑤处应填（ ）。

- A. ok && c[s] == c[t]
- B. ok && c[s] != c[t]
- C. !ok || c[s] == c[t]
- D. !ok && c[s] != c[t]

信奥陈老师

【解析】进入这一分支说明已保证正负边都存在。输出 0 的两种情形：① !ok：存在奇环，任意奇偶性的走法都能实现，可把权值调成 0；② c[s] == c[t]：所有 $s \to t$ 路线的边数都是偶数，权值和必为偶数，配合成对调整即可到 0。反之（二分图且 c[s] != c[t]）边数必为奇数，权值必为奇数，最小只能是 1。正确选项：C

## （2）标准答案

有 $n$ 名学生参加一次考试，考试共有 $m$ 道选择题，每道题只有 A、B 两个选项。

第 $i$ 名学生的作答用一个长度为 $m$ 的字符串 $a_i$ 表示，若最终公布的标准答案与该学生在某道题上的作答相同，则该学生得 1 分，否则不得分。记第 $i$ 名学生最终得到的总分为 $r_i$。给定每名学生的目标分数 $x_i$。现在需要构造一份标准答案，使得 $\sum_{i=1}^{n} |r_i - x_i|$ 尽可能大。

数据满足 $1 \le n \le 20,\ 1 \le m \le 200,\ 0 \le x_i \le m$。

以下程序从枚举符号的角度处理 $\sum_{i=1}^{n} |r_i - x_i|$，把它写成更易优化的形式。

对于非非零整数 $x$，`__builtin_ctzll(x)` 返回 $x$ 的二进制表示末尾连续 0 的个数。

`__builtin_popcountll(x)` 返回 $x$ 的二进制表示中 1 的个数。

程序输出一组满足答案的标准答案，请补全程序。

```
#include <cstdlib>
#include <iostream>
#include <string>
#include <vector>

using namespace std;
typedef long long ll;
typedef unsigned long long ull;

int main() {
    int n, m;
    cin >> n >> m;
    vector<ll> x(n), c(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i];
        c[i] =  ① ;
    }
    vector<string> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    vector<int> s(n, -1);
    vector<ll> q(m, 0);
    ll C = 0, S = 0;
    for (int i = 0; i < n; i++) {
        C -= c[i];
        for (int j = 0; j < m; j++) {
            if (a[i][j] == 'A') q[j]--;
            else q[j]++;
        }
    }
    for (int j = 0; j < m; j++) S += abs(q[j]);
```

信奥陈老师

```
    ll ans = C + S;
    ull best = 0, lst = 0;
    for (ull mask = 1; mask < (1ULL << n); mask++) {
        ull g =  ② ;
        ull d = g ^ lst;
        int k =  ③ ;
        C -=  ④ ;
        for (int j = 0; j < m; j++) {
            ll old = q[j];
            int v = (a[k][j] == 'A' ? 1 : -1);
            q[j] -= 2 * 11 * s[k] * v;
            S += abs(q[j]) - abs(old);
        }
        s[k] = -s[k];
        if (C + S > ans) {
            ans = C + S;
            best = g;
        }
        lst = g;
    }

    for (int i = 0; i < n; i++) {
        if (best >> i & 1) s[i] = 1;
        else s[i] = -1;
    }
    string res(m, 'A');
    for (int j = 0; j < m; j++) {
        ll v = 0;
        for (int i = 0; i < n; i++) {
            if (a[i][j] == 'A') v += s[i];
            else v -= s[i];
        }
        if ( ⑤ ) res[j] = 'A';
        else res[j] = 'B';
    }
    cout << res << endl;
    return 0;
}
```

信奥陈老师

给定 $n$ 名学生的作答（长度 $m$ 的 A/B 串）与预期得分 $x_i$，构造标准答案使 $\sum_{i=1}^{n}|r_i-x_i|$ 最大，其中 $r_i$ 是第 $i$ 名学生的实际得分。

**程序先读明白**：核心是绝对值拆解

$$|r_i-x_i|=\max_{\sigma_i\in\{+1,-1\}}\sigma_i(r_i-x_i)$$

于是目标化为 $\max_{\sigma,t}\left[\sum_i\sigma_i r_i-\sum_i\sigma_i x_i\right]$。把 $r_i=\sum_j[a_i[j]=t_j]$ 代入（`A` 记 $+1$、`B` 记 $-1$），可整理成

$$\sum_i\sigma_i\left(\frac{m}{2}-x_i\right)+\frac{1}{2}\sum_j t_j\left(\sum_i\sigma_i b_{ij}\right)$$

两边同乘 2 并记 $c_i=m-2x_i$、$q_j=\sum_i s_i b_{ij}$，目标就是 $C+S=\sum_i s_i c_i+\sum_j|q_j|$：对每个 $j$ 取 $t_j=\mathrm{sign}(q_j)$ 即得该列的 $\sum_j|q_j|$。程序用**格雷码**枚举全部 $2^n$ 个 $\sigma$，每次只翻一位并增量更新 $C$、$S$。

**第 39 题** ①处应填（ ）。

- A. `2 * x[i] - m`
- B. `-m + 2 * x[i] + 1`
- C. `m - 2 * x[i]`
- D. `m + 2 * x[i]`

【解析】上面对目标乘 2 后，$x_i$ 的系数是 $m-2x_i$，且程序里 `C -= c[i]` 配合 `s[i]` 初值 $-1$，正好对应 $C=\sum_i s_i c_i$，故 `c[i] = m - 2 * x[i]`。A 符号相反；B、D 多出常数或符号错误。
正确选项：C

**第 40 题** ②处应填（ ）。

- A. `mask | (mask >> 1)`
- B. `mask ^ (mask >> 1)`
- C. `mask & (mask >> 1)`
- D. `mask ^ ((mask >> 1) + 1)`

【解析】相邻两个 $g$ 只差一位才能增量更新，这正是**格雷码** $g=mask\oplus(mask\gg1)$。于是 `d = g ^ lst` 恰好只有一位为 1。正确选项：B

**第 41 题** ③处应填（ ）。

- A. `__builtin_ctzll(d) + 1`

- B. `__builtin_popcountll(d)`
- C. `__builtin_ctzll(g)`
- D. `__builtin_ctzll(d)`

【解析】`d = g ^ lst` 是本次翻转的那一位，`__builtin_ctzll` 返回末尾连续 0 的个数，正好等于该位的下标，即被翻转的是第 $k$ 个学生。A 多了 1（下标错位）；B 返回 1（恒为 1，无用）；C 取 `g` 的末尾 0 个数，与翻转位无关。正确选项：D

**第 42 题** ④处应填（ ）。

- A. `2ll * s[k] * c[k]`
- B. `s[k] * c[k]`
- C. `2ll * (s[k] - c[k])`
- D. `2ll * c[k]`

【解析】`s[k]` 从 $-1$ 翻到 $+1$，$C=\sum_i s_i c_i$ 的增量为 $(+1-(-1))c_k=2c_k$，而当前 `s[k]` 仍为旧值 $-1$，故写 `C -= 2ll * s[k] * c[k]` 恰好等于 `C += 2c_k`。B 少乘 2；C、D 形式不符。正确选项：A

**第 43 题** ⑤处应填（ ）。

- A. `v >= (n & 1)`
- B. `v > (n & 1)`
- C. `v + (n & 1) >= 0`
- D. `v * (n & 1) >= 0`

---

【解析】$t_j=\text{sign}(q_j)$，即 `v > 0` 时该列取 A、`v < 0` 时取 B，`v = 0` 时两者等价。注意 $v=\sum_i s_i b_{ij}$ 是 $n$ 个 $\pm1$ 之和，**奇偶性与 $n$ 相同**：$n$ 为偶数时 $v$ 为偶数，$n$ 为奇数时 $v$ 为奇数。因此 `v >= (n & 1)` 恰好等价于 `v > 0`：$n$ 偶时它是 `v >= 0`，$n$ 奇时它是 `v >= 1`。B 在 $v=0$ 且 $n$ 偶时会误判；C、D 写法不能表达符号判断。正确选项：A